

NIDS

Network Intrusion Detection System

Week 1 — Study Notes & Project Log

Ayush · NIT Hamirpur · ECE
Campus Placement Portfolio Project
D3.js · React · Node.js · PostgreSQL · Socket.io

1. What is This Project?

A browser-based Network Intrusion Detection System (NIDS) that visually simulates how a real computer network works, detects suspicious traffic using graph algorithms, and shows real-time alerts on a dashboard.

Simple Analogy

- Your network = an office building
- Every device (PC, server, router) = a room
- Data packets = letters being delivered between rooms
- Your app = the security guard system watching all movement
- Graph algorithms = the logic the guard uses to detect intruders

One-Line Pitch

"A visual security guard for a computer network — it watches all traffic, spots anything suspicious, blocks threats, and shows everything live on a dashboard."

2. Tech Stack

Layer	Technology
Frontend	React.js + D3.js + Chart.js + Tailwind CSS
Backend	Node.js + Express.js
Database	PostgreSQL via Prisma ORM
Real-time	Socket.io
Auth	JWT + bcrypt
Hosting	Vercel + Render + Supabase

3. Five-Week Build Timeline

Week	Goal	Status
Week 1	Network canvas with drag-drop nodes using D3.js	DONE
Week 2	Packet simulation + BFS anomaly detection algorithm	NEXT
Week 3	Real-time alerts via Socket.io + live alert dashboard	
Week 4	Firewall rules engine + centrality visualization	
Week 5	Polish UI + deploy + write README	

4. Graph Algorithms Used

BFS — Breadth First Search

Checks every device level by level from the entry point. Used to detect unusual traffic by comparing the actual packet path to expected normal paths.

Analogy: A security guard checking every room floor by floor.

Dijkstra's Algorithm

Finds the lowest-risk path for a packet to travel from source to destination across the network graph.

Analogy: GPS finding the safest route, not just the shortest.

Betweenness Centrality

Calculates which device has the most traffic passing through it. The device with the highest score is the biggest attack target.

Analogy: The most important junction in a city road network.

Union-Find

Groups all infected/breached devices into clusters. When one device is compromised, all connected devices are automatically grouped for quarantine.

Analogy: Finding all buildings sharing the same contaminated water pipe.

5. How D3.js Draws the Canvas

What is SVG?

SVG = Scalable Vector Graphics. It's like drawing on paper inside your browser. Instead of pixels, you draw shapes using code. D3.js is a JavaScript library that creates and controls these SVG elements.

```
<svg width="400" height="400"> <circle cx="200" cy="200" r="50"
fill="blue"/> </svg>
```

Step 1 — Selecting the SVG

We grab the SVG element and store it as our drawing board:

```
const svg = d3.select('#canvas')
```

d3.select finds the HTML element with id='canvas'. Everything gets drawn inside it.

Step 2 — Creating Groups (Layers)

```
const edgeGroup = svg.append('g').attr('class','edges') const nodeGroup =
svg.append('g').attr('class','nodes')
```

Groups are like folders — edges are drawn first (bottom layer), nodes on top. This ensures lines appear behind circles.

Step 3 — The D3 Data Join

D3's superpower — it compares your data array vs what's already drawn:

```
const link = edgeGroup.selectAll('.link').data(edges, d => d.id)
link.enter().append('line') // NEW edges → draw a line .merge(link) // combine new +
existing .attr('x1', d => getNode(d.source).x) .attr('y1', d =>
getNode(d.source).y) .attr('x2', d => getNode(d.target).x) .attr('y2', d =>
getNode(d.target).y) link.exit().remove() // deleted edges → remove line
```

Step 4 — Drawing Each Node

Each node is a group of 4 SVG elements stacked together:

```
<g transform="translate(300, 200)"> // positioned at 300,200 <circle r="24"
fill="#0d2a6e"/> // dark circle <text y="1">laptop emoji</text> // icon
<text y="38">PC-1</text> // label <text
y="49">192.168.0.1</text> // IP address </g>
```

Step 5 — Packet Animation

The moving dot transitions smoothly from node to node:

```
dot.transition().duration(500) // 500ms per hop .attr('cx', nextNode.x) .attr('cy',
nextNode.y) .on('end', () => nextHop()) // when done, go to next node
```

6. How Drag-Drop Works

Drag = 3 events happening in sequence:

- mousedown → drag starts (user presses mouse button)
- mousemove → dragging (user moves mouse while holding)
- mouseup → drag ends (user releases mouse button)

Attaching Drag Behaviour

```
d3.drag().on('start', dragStart).on('drag', dragged).on('end', dragEnd)
```

This creates a drag behaviour object and attaches it to every node group using `.call()`

dragStart

```
function dragStart(event, d) { if (mode !== 'add') return // only drag in add mode  
d3.select(this).raise() // bring node to front }
```

`d3.select(this).raise()` moves this node's SVG group to the top so it appears above other nodes while dragging.

dragged — The Key Function

```
function dragged(event, d) { d.x = Math.max(30, Math.min(W-30, event.x)) d.y =  
Math.max(30, Math.min(H-30, event.y)) render() }
```

`event.x` and `event.y` = current mouse position. We update the node's stored position to match. `Math.max/min` clamps the position so nodes can't go off the canvas edge. Then `render()` redraws everything.

Key insight: We don't move the SVG element directly. We update the data (`d.x`, `d.y`), then `render()` redraws from scratch. This is why edges follow the node automatically — they always read the node's latest `x,y` from the data.

The Complete Flow

```
1. User clicks canvas 2. svg.on('click') fires → addNode(x, y) 3. addNode() pushes new  
object into nodes[] array 4. render() is called 5. D3 reads nodes[] → draws new circle +  
label + icon 6. User drags the node 7. dragged() fires → updates d.x, d.y → render() 8.  
D3 moves the group via translate(newX, newY) 9. Edges redraw because they read  
getNode().x / .y
```

7. API Routes (Week 2 Preview)

Method	Route	Purpose
POST	/api/network/save	Save topology to DB
GET	/api/network/:id	Load topology from DB
POST	/api/simulate/packet	Send a test packet
POST	/api/simulate/attack	Simulate an intrusion
GET	/api/alerts	Fetch all alerts
POST	/api/firewall/rule	Add a firewall rule
DELETE	/api/firewall/rule/:id	Remove a firewall rule
GET	/api/traffic/live	Live traffic (Socket.io)

8. Interview Pitch

"I built a browser-based Network Intrusion Detection System that models a live network as a graph. It uses BFS for anomaly detection, Dijkstra's algorithm for safe path finding, Betweenness Centrality to identify high-risk nodes, and Union-Find to cluster breached devices. Real-time alerts are powered by Socket.io, and users can simulate attacks, trace malicious paths, and manage firewall rules — all visualized on an interactive D3.js canvas built with the MERN stack and PostgreSQL."

NIDS Project — Week 1 Complete

Next: Week 2 — Node.js Backend + BFS Algorithm

Ayush · NIT Hamirpur · ECE · Campus Placement Portfolio